Concept Drift Adaptation by Exploiting Drift Type

JINPENG LI, HANG YU, ZHENYU ZHANG, XIANGFENG LUO, and SHAORONG XIE, School of Computer Engineering and Science, China

Concept drift is a phenomenon where the distribution of data streams changes over time. When this happens, model predictions become less accurate. Hence, models built in the past need to be re-learned for the current data. Two design questions need to be addressed in designing a strategy to re-learn models: which type of concept drift has occurred, and how to utilize the drift type to improve re-learning performance. Existing drift detection methods are often good at determining when drift has occurred. However, few retrieve information about how the drift came to be present in the stream. Hence, determining the impact of the type of drift on adaptation is difficult. Filling this gap, we designed a framework based on a lazy strategy called Type-Driven Lazy Drift Adaptor (Type-LDA). Type-LDA first retrieves information about both how and when a drift has occurred, then it uses this information to re-learn the new model. To identify the type of drift, a drift type identifier is pre-trained on synthetic data of known drift types. Furthermore, a drift point locator locates the optimal point of drift via a sharing loss. Hence, Type-LDA can select the optimal point, according to the drift type, to re-learn the new model. Experiments validate Type-LDA on both synthetic data and real-world data, and the results show that accurately identifying drift type can improve adaptation accuracy.

CCS Concepts: • **Information systems → Data stream mining;**

Additional Key Words and Phrases: Concept drift, data streams, drift detection, drift adaptation

ACM Reference Format:
Jinpeng Li, Hang Yu, Zhenyu Zhang, Xiangfeng Luo, and Shaorong Xie. 2024. Concept Drift Adaptation by Exploiting Drift Type. *ACM Trans. Knowl. Discov. Data.* 18, 4, Article 96 (February 2024), 22 pages. https://doi.org/10.1145/3638777

1 INTRODUCTION

Concept drift is a phenomenon where the data distribution of a target domain varies arbitrarily over time [13, 36, 46]. Formally, consider a distribution $P(X, y)$ that evolves as time passes, where X are the feature vectors, and y are the labels. According to the principles of machine learning, changes in the data distribution $P(X, y)$ will cause the predictions to become less accurate over time [42, 45, 47]. Hence, a predictor built in the past needs to be re-learned for the current data. The aim, when re-learning a predictor, is to find the optimal point of drift; i.e., the time point at which replacing the old predictor with a new re-learned predictor gives the best final result. However, two design questions need to be addressed in designing a strategy to find the optimal

This research work was funded by National Natural Science Foundation of China Youth Fund (No. 62302287), Project supported by the Shanghai Committee of Science and Technology, China (Grant No. 23ZR1423500), the work presented in this article was supported by the Shanghai Yangfan Program (22YF1413600).
Authors' address: J. Li, H. Yu, Z. Zhang, X. Luo, and S. Xie, School of Computer Engineering and Science, Shanghai University, No. 99 Shangda Road, Baoshan District, Shanghai, 200444, China; e-mails: {lijinpeng, yuhang, zhenyuzhang, luoxf, srxie}@shu.edu.cn.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1556-4681/2024/02-ART96
https://doi.org/10.1145/3638777

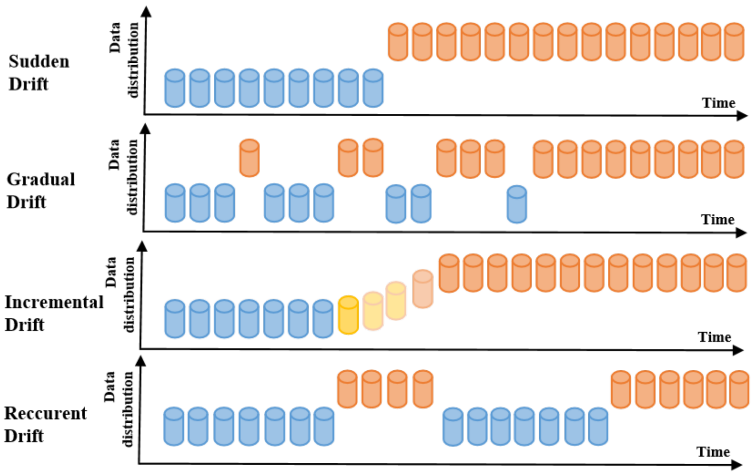


Fig. 1. Types of concept drift [13].

point: which type of concept drift has happened, and how to utilize the drift type to improve re-learning performance.

It is commonly accepted that there are four types of concept drift (see Figure 1) [13, 23, 52]: (1) sudden drift, where a new concept suddenly appears within a short time; (2) gradual drift, where a new concept gradually replaces an old one over a period of time; (3) incremental drift, where an old concept becomes a new concept incrementally in a period of time; and (4) reoccurring drift, where an old concept reoccurs after a period of time.

Over the past 20 years, many methods have been proposed to detect concept drift. However, in general, all follow the same basic four-step framework [2, 23, 51]: (1) data retrieval, where chunks of data are retrieved from a data stream; (2) data modeling, where the retrieved data is abstracted and the key features containing sensitive information are extracted, (3) test statistics calculation, where dissimilarity is measured or distances are estimated; and (4) hypothesis tests, where a particular hypothesis test is used to evaluate the statistical significance or p-value of the change observed in Step 3.

However, while such frameworks are good at detecting information about the time at which the concept drift information occurred [8], they cannot directly identify how the drift occurred; i.e., the type of drift [13], such as sudden drift. The reason for this is that, even though a test statistic can detect the point of change in the data distribution, e.g., the average error rate, determining how the drift occurred relies on the changing patterns in the data distribution, which is the relationship between the test statistics in adjacent timestamps [41].

If the type of concept drift can be acquired, we can go on to discuss the relationship between the type of drift and the drift adaption. Different adaptation methods can be implemented for different kinds of concept drift. Note that, in this work, our focus is on detecting immediate types of concept drift. Hence, we do not consider the reoccurring type where an old concept may occur again. Instead, we use the term normal to represent no concept drift.

In this article, to discuss the impact of the drift type on drift adaptation, we developed a framework based on a lazy strategy [23] called **Type-Driven Lazy Drift Adaptor** (**Type-LDA**). Type-LDA first retrieves information about how a drift has occurred, and then uses this information to re-learn a new model. However, different from existing **drift detection methods** (**DDM**), the drift detection in Type-LDA is trained in the offline manner; specifically, the basic idea is to learn

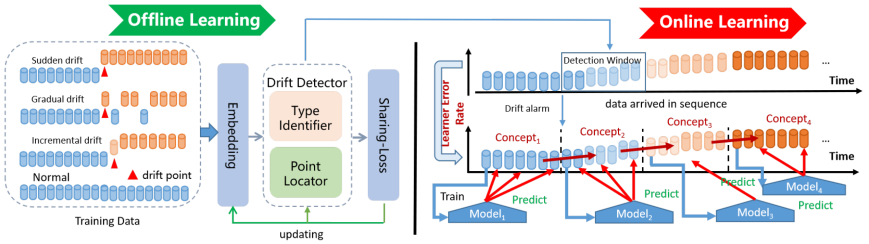


Fig. 2. The flowchart of Type-LDA. Type-LDA consists of offline learning and online learning. (1) The offline learning process is to train the Drift Detector with data stream and update the network parameters through Sharing-Loss method. (2) The online learning process is based on the trained Drift Detector to detect the concept drift in the new data stream, so as to improve the prediction performance of the Model_i, $i \in (1, 2, \ldots, n)$.

the error rates of each type of concept drift and form a prototype vector in the offline training process. Therefore, identifying the type of drift can be seen as locating the prototype vector of each category in the semantic space, similar to a classification issue. In addition, as the prototype vector also represents the different characteristics of data distribution before and after drift, the prototype vector also has important information to support the identification of the optimal drift point. However, as the drift point is a time point, predicting the drift point is thought of as a regression task, so we have integrated the prototype vector into a regression model to predict the optimal drift point. Meanwhile, we have designed a sharing-loss function that can train two task models together. In addition, an active learning is used to fine-tune the model on the new data stream to improve the model's reliability further (see Figure 2). This article makes the following four novel contributions:

— We propose a novel framework, which adopts an end-to-end method without considering any statistical test, for detecting concept drift with an offline training process. Consequently, the framework can not only detect the "when" of concept drift but also the "how", i.e., the type of drift, which enables investigation of the impact of the drift type on drift adaptation.
— We propose a prototypical-network-based model for identifying the type of concept drift and a regression model to identify when the concept drift has occurred. In addition, we designed a sharing-loss function to help learn two models at the same time.
— We suggest a **stream-based active learning** method (**SAL**), which can recognize and adapt to various types of concept drift by adapting Type-LDA to underlying distributions in the stream, to improve the precision of prediction.
— Comparative experiments demonstrate that our method is superior to **SOTA** (**State-of-the-art**) drift adaptation methods with a simple strategy, which proves the basic idea of our method: that identifying which type of concept drift has occurred is helpful for finding the optimal point.

This article is arranged as follows. Related work is presented in Section 2. A detailed description of the proposed Type-LDA framework is given in Section 3. Section 4 presents the experiments, which involve several popular datasets, and Section 5 summarizes the article along with our intentions for future work.

2 RELATED WORK

A review of the pertinent literature revealed that drift adaptation techniques can be broadly categorized into two groups: ensemble-based methods and lazy strategy-based methods.

2.1 Ensemble-Based Drift Adaptation Methods

The core idea of using ensemble methods to deal with concept drift is to retain and reuse the old model in the case of repeated drift to save the resource investment of retraining the new model on repeated concepts.

Bagging, Boosting, and Random Forests are used to process streaming data with conceptual drift. An online version of the bagging method was first proposed in [27], which uses each instance only once to simulate the batch mode bagging. An adaptive enhancement methodology was developed [26], which uses a hypothesis test to monitor the accuracy of predictions and assumes that the classification error of non-drift data conforms to a Gaussian distribution. An adaptive random forest algorithm is provided in [32] that extends the random forest tree algorithm using concept DDMs to determine when to replace old trees with new ones. Similarly, [38] uses the Hoeffding bound to distinguish concept drift from noise within decision trees.

There are also many new ensemble methods that use new voting techniques to handle concept drift. One example is **Dynamic Weighted Majority** (**DWM**) [24], an ensemble approach that accommodates drift through a simple set of weighted voting rules, managing basic classifiers based on the performance of individual classifiers and global ensembles. The ensemble method Learn++NSE [8] alleviates the above problem by weighting the classifier based on the prediction error rate of the basic classifier on the new data; that is, when the error rate of the latest classifier exceeds 50 %, the new classifier is trained based on the latest data.

Research on ensemble approaches to dealing with certain types of concept drift is also ongoing. For example, Accuracy Update Ensemble [1] for sudden and gradual drift is a batch-mode-weighted voting ensemble method based on incremental base classifiers. By re-weighting, the ensemble can react quickly to sudden drift. The Weights Adjustment method [36] handles sudden drift and gradual drift by using an ensemble for constructing different conceptual drift types using weighted instances and weighted classifiers.

2.2 Lazy Strategy-Based Drift Adaptation Methods

Perhaps the most direct way to deal with concept drift is to retrain a new model with the latest data to replace the outdated model, called lazy strategy. Therefore, a drift detector displaying the concept is needed to determine the time to retrain the model.

DDM [11] was the first algorithm to contain a defined warning and drift levels to signal concept drift. Many subsequent algorithms have adopted a similar implementation, e.g., **Early Drift Detection Method** (**EDDM**) [5], **Hoeffding's inequality-based Drift Detection Method** (**HDDM**) [10], **Fuzzy Windowing Drift Detection Method** (**FW-DDM**) [22], and **Dynamic Extreme Learning Machine** (**DELM**) [41]. In contrast to DDM and other similar algorithms, the **Statistical Test of Equal Proportions Detection** (**STEPD**) [25] detects changes in error rates by comparing the most recent time window with the overall time window. For each timestamp, there are two-time windows in the system. However, the size of the new window must be defined by the user. Another popular two-time-windows-based drift detection algorithm is **Adaptive Windowing** (**ADWIN**) [6]. Unlike STEPD, ADWIN does not require users to define the size of the compared windows in advance; it only needs users to specify the total size n of a "sufficiently large" window W. Guo et al. [14] proposed concept drift type identification based on multi-sliding windows. During the detection process, drift length is determined through the growth of an accompanying window, and the identification of the type of drift is based on the detected drift length. Wang et al. [39] proposed Online Feature Screening for Data Streams with the Concept Drift method.

Instance-based lazy learners for handling concept drift have also been studied intensively. Real-Time Adaptive Classifier [46, 50] is such a method that follows the "detect and update model" strategy. It extends the traditional CUSUM test [44] into a PDF-free form to handle drift detection,

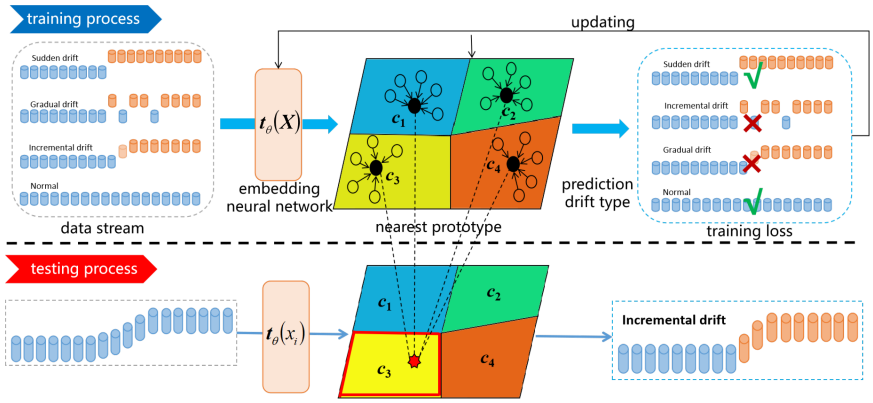


Fig. 3. Illustration of Type-LDA's drift type identifier module.

and then integrates this detection method with the KNN classifier. If concept drift is detected, the old instance is removed from the case base. The authors of [53] and [15] extend the algorithm by calculating and comparing the current concept with the previously stored concept to deal with recurring concepts. The step-wise redundancy removal algorithm is adopted to ensure that the processed instance library can still retain enough information for future drift detection.

3 TYPE-DRIVEN LAZY DRIFT ADAPTOR

In this article, to investigate the impact of the drift type on drift adaptation, we use a lazy strategy [23] for drift adaptation. Specifically, we design a drift detector that will retrieve information about both how a drift has occurred and which time point is the optimal drift point. Then, we will start to re-learn a new predictor at the optimal drift point to replace the old predictor.

3.1 Drift Type Identifier

The basic idea of our proposed drift adaption strategy is that the performance of the drift detection will directly influence the effect of the drift adaption. Hence, we need to design a method for detecting concept drift at first (As shown in Figure 3).

Almost all researchers think concept drift can only be detected in real-time, and all follow the same basic four-step framework [23]. However, the efficacy of this framework depends on how the test statistic of the samples is calculated [48]. As a result, the detection process is sensitive to noise, in that noise can impact the statistic properties [30], such as mean or variance.

To overcome the problem, in this article, we propose a novel framework that adopts an end-to-end method, without considering any statistical test, for detecting concept drift with an offline training process. Based on the intuitions, we first utilize the **Massive Online Analysis (MOA)** tool introduced in https://moa.cms.waikato.ac.nz/tutorial-5-simulating-concept-drift-in-moa/ to generate L data streams that has multiple types of concept drift. It is the most common tool for generating synthetic data streams. In the MOA, gradual or sudden drift data streams are created by the SEA generator [3], and incremental drifts or no drifts ones are created by the HYP generator [16]. We directly apply a prediction model to a data stream to get error rate e_t at timestamp t, e.g., naive Bayes or decision tree. Furthermore, we utilize a n length window W_i, which means that W_i contains n samples. As a result, the average error rate $\hat{e}_t$ of W_i is

$$\hat{e}_t = \frac{\sum_{t=1}^{n} e_t}{n}. \tag{1}$$

At the same time, we calculate the distance between $\hat{e}_t$ and $\hat{e}_{t+1}$ as follows:

$$gap_i = \hat{e}_{t+1} - \hat{e}_t. \tag{2}$$

Suppose each data stream contains $l \times n$ samples, then l is windows count, and $l - 1$ is the number of gap_i can be obtained. Therefore, a sample in the pre-training phase can be represented as $\{X_i, y_i\} \in \mathbb{R}^l$, where $X_i = \{gap_1, \ldots, gap_{l-1}\}$, y_i is the label and $y_i \in \{1, \ldots, 4\}$. As a result, the underlying data distribution of a data stream is mapped to $\{X_i, y_i\}$.

Note that our framework allows for flexible selection of error features X_i. However, to pay more attention to our contributions, based on our empirical findings, we only employ straightforward models and tumbling windows throughout the experiments, producing a performance that is reasonable. The question of how to better design the transferable information will be left to future work to improve the Type-LDA framework.

As mentioned, learning the drift type identifier in our research can be considered as a classification issue. However, as the MOA method we used in this article cannot generate a rich supply of data, learning the drift type identifier is considered as a few-shot learning problem. As a result, to keep the generalization of the drift type identifier, it is built based on a prototypical neural network because the prototype network [35] is a more practical and effective method to deal with the problem of few-shot multi-class learning problem.

ALGORITHM 1: Training Drift Type Identifier.

Input : Data stream $G : \{(X_1, y_1), \ldots, (X_L, y_L), L > 0\}$, and a set of data streams labeled k: D_k.
Parameter: The number of support data stream N_s, the number of query data stream N_q. RANDOMSAMPLE(D, N) represents uniformly choosing a set of N elements from set D at random. t_θ denotes data stream X_i maps to embedding space vector by embedding neural network. p_θ means to find the nearest prototype c_k of X_i in the embedding space.
Output : The training loss of $J(\theta)$.

$J(\theta) \leftarrow 0$
for k *in* $\{1, \ldots, 4\}$ **do**
  $S_k \leftarrow$ RANDOMSAMPLE(D_k, N_s)
  $Q_k \leftarrow$ RANDOMSAMPLE(D_k, N_q)
  $c_k = \frac{1}{|S_k|} \sum_{(X_i, y_i) \in S_k} t_\theta(X_i)$
  for (X, y) *in* Q_k **do**
    $p_\theta(y = k \mid X) = \frac{\exp(-d(t_\theta(X), c_k))}{\sum_{k'} \exp(-d(t_\theta(X), c_{k'}))}$
    $J(\theta) += -\frac{1}{4N_q} \log p_\theta(y = k \mid X)$
  end
end
return $J(\theta)$

When given a few samples and a metric function, classification problems are seen in a prototypical network as locating the prototype vector of each category in the semantic space. Then, the category of the new samples is predicted to be the category of their nearest neighbors. Hence, based on the training dataset $G : \{(X_1, y_1), \ldots, (X_L, y_L), L > 0\}$, there are two steps in the prototypical neural network. First, an embedding neural network t_θ with a learnable parameter θ maps the error-features X_i into the embedding space $t_\theta(X_i)$; then, we calculate the average of its support set in the hidden space for each class k as a single prototype vector c_k:

$$c_k = \frac{1}{|S_k|} \sum_{(X_i, y_i) \in S_k} t_\theta\left(X_i\right), \tag{3}$$

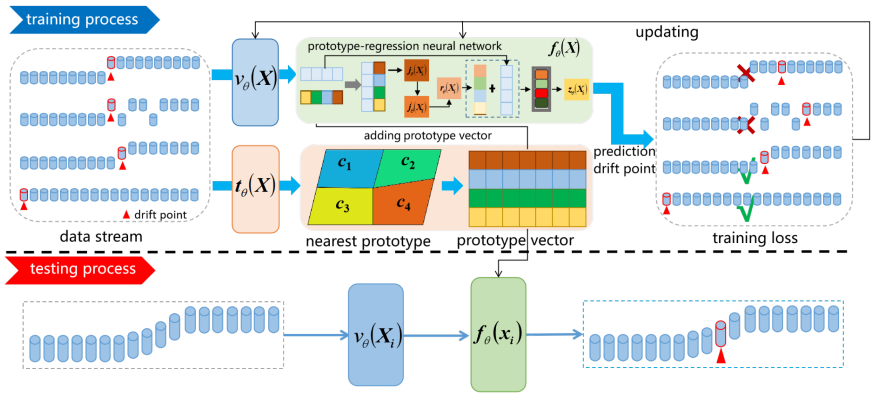


Fig. 4. Illustration of Type-LDA's drift point locator module.

where S_k is the set of data streams that labeled k, $|S_k|$ is the number of support sets. Suppose we have a distance function d, to determine, say, a Euclidean distance, classification for a query point X is then carried out by locating the closest prototype c_k over a gap in the embedding space:

$$p_\theta\left(y=k \mid X_i\right)=\frac{\exp\left(-d\left(t_\theta\left(X_i\right), c_k\right)\right)}{\sum_{k'} \exp\left(-d\left(t_\theta\left(X_i\right), c_{k'}\right)\right)}. \tag{4}$$

Learning proceeds by minimizing the negative log-probability $J(\theta)=-\log p_\theta(y=k \mid X_i)$ of the true class k via **stochastic gradient descent** (**SGD**).

In the training phase, as delineated in Algorithm 1, the procedure involves selecting N_s data streams at random for each category of concept drift within each episode. These selected data streams correspond to the prototype vector c_k within the embedding space, with their values being determined by the average of N_s. Subsequently, a set of N_q support samples, each annotated with labels, is randomly drawn from the data stream. These support samples are employed to gauge the accuracy of the learned concept drift type identifier. Lastly, the training loss is assessed to fine-tune the embedding neural network. In the event of encountering a new data stream during the inference phase, the nearest prototype center is chosen during the reasoning stage. This choice relies on both the learned prototype center c_k and the embedding space, facilitating the estimation of the class. Algorithm 1 provides a pseudo-code representation of the model training process. In the context of inference, when confronted with a novel concept drift feature, the approach involves selecting the nearest prototype representation center to ascertain the type of concept drift.

3.2 Drift Point Locator

This subsection discusses how to predict the time of drift. As is known to all, a data stream changes with time t and only one data point arrives at each moment. Thus, the drift time t is defined as the index coordinate of the drift point. Therefore, learning the drift point locator should be considered one of the regression issues.

In our drift point prediction task, we assume L training data stream $D:\{(X_1, z_1), \ldots, (X_L, z_L), L>0\}$, where X_i is also the error-features with the corresponding drift point label z_i and $z_i \in \{1, \ldots, l-1\}$. There are two steps in drift point locator, as shown in Figure 4. First, an embedding neural network $v_\theta \in \mathbb{R}^{d \times N}$ with a learnable parameter θ maps the error-features X_i into the embedding space $v_\theta(X_i)$:

$$v_\theta\left(X_i\right)=\ \mathrm{ReLU}\left(W_e X_i+b_e\right), \tag{5}$$

where $W_e \in \mathbb{R}^{d\times d}$, $b_e \in \mathbb{R}^{d\times N}$ are trainable parameters, d is the feature dimension, N is the number of the trained data streams.

However, in our opinion, the drift points of different types of concept drift have different characteristics, so it is necessary to consider the information on drift type while locating the drift point. In the above section, we know a single prototype $c_k \in \mathbb{R}^{d\times N}$ can be viewed as the most representative information for each class k. As a result, by combining the prototype vector c_k and the encoding vector v_θ, we can get a new feature vector:

$$j_\theta(X_i) = \text{ReLU}\left(W_c\left[c_k(X_i), v_\theta(X_i)\right] + b_c\right), \tag{6}$$

where $W_c \in \mathbb{R}^{d\times 2d}$ and $b_c \in \mathbb{R}^{d\times N}$ are trainable parameters. Although the prototype vector contains the features of each type, too much worthless information may be learned in training process, leading to the instability of the model. To ensure the stability of training, we add $v_\theta(X_i)$ again so that the model can select the optimal training features:

$$j'_\theta(X_i) = \text{ReLU}\left(W_p j_\theta(X_i) + b_p\right), \tag{7}$$

$$r_\theta(X_i) = \text{ReLU}\left(W_a\left[j'_\theta(X_i) + v_\theta(X_i)\right] + b_a\right), \tag{8}$$

where $W_p \in \mathbb{R}^{d\times d}$, $W_a \in \mathbb{R}^{d\times d}$, $b_p \in \mathbb{R}^{d\times N}$ and $b_a \in \mathbb{R}^{d\times N}$ are trainable parameters. $j'_\theta(X_i)$ is obtained from a dimension transformation of a $j_\theta(X_i)$ characteristic matrix, so as to make the dimension of $j'_\theta(X_i)$ and $v_\theta(X_i)$ consistent.

A regression model is used as the drift point locator, the structure of the regression model is the same as that of a classical Multi-layer Perception[28]:

$$\hat{z}_\theta(X_i) = W_o r_\theta(X_i) + b_o, \tag{9}$$

where $r_\theta(X_i)$ is the embedding space vector that combines prototype vector and input data. $W_o \in \mathbb{R}^{1\times d}$ and $b_o \in \mathbb{R}^{1\times N}$ are trainable parameters. Then, given a set of error-features in data stream X_i, each drift point of every data stream corresponds to one object value. Hence, the learning objective is to minimize the following **Mean-Square-Error** (**MSE**) loss L on all drift points of the error-features X_i that exist during training:

$$L(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left\|z_\theta(X_i) - \hat{z}_\theta(X_i)\right\|^2, \tag{10}$$

where z_θ is the real value of drift point, and $\hat{z}_\theta$ is regression model output value.

Algorithm 2 is presented to outline the pseudo-code for training the model. The drift point locator takes a data stream X_i as input and produces output indicating the locations of drift occurrences within the data stream. The learning process of the drift point locator involves the utilization of SGD to minimize the average absolute error (MAE) of the ground truth values denoted as $L(\theta)$.

3.3 Sharing-Loss Function

In this section, we design a multi-task loss function by maximizing the Gaussian likelihood [18] and apply it to the training of Type-LDA. Let $H^1_\theta(X_i)$ be the output of the drift type identifier with weights θ on input X_i. For classification tasks, a softmax function is used to squash the output of a probabilistic model and sample from the resulting probability vector:

$$p\left(\hat{y}_1 = k \mid H^1_\theta(X_i)\right) = \text{softmax}\left(H^1_\theta(X_i)\right). \tag{11}$$

Let $H^2_\theta(X_i)$ be the output of the drift point locator with weights θ on input X_i. For regression tasks, we use a Gaussian distribution to define the likelihood with the mean based on the model

ALGORITHM 2: Training Drift Point Locator.

Input : data steam $D : \{(X_1, z_1), \ldots, (X_L, z_L), L > 0\}$, where the corresponding label z_i and z_i are the real value in range of $[1, l-1]$.

Parameter: v_θ is data stream (X) embedding vector, j_θ is the fusion vector by v_θ and prototype vector f_θ, j'_θ is obtained by dimension transformation of j_θ, r_θ is obtained by learning of j'_θ and v_θ, z_θ is the output of the regression model, c_k is prototype vector, N is the number of the trained data streams.

Output : the training loss of $L(\theta)$.

for i *in* $\{1, \ldots, N\}$ **do**

$v_\theta(X_i) = \text{ReLU}(W_e X_i + b_e)$

$j_\theta(X_i) = \text{ReLU}(W_c[c_k(X_i), v_\theta(X_i)] + b_c)$

$j'_\theta(X_i) = \text{ReLU}(W_p j_\theta(X_i) + b_p)$

$r_\theta(X_i) = \text{ReLU}(W_a[j'_\theta(X_i) + v_\theta(X_i)] + b_a)$

$\hat{z}_\theta(X_i) = W_o r_\theta(X_i) + b_o$

end

$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \| z_\theta(X_i) - \hat{z}_\theta(X_i) \|^2$

output:

$$p\left(\hat{y}_2 \mid H_\theta^2(X_i)\right) = H_\theta^2(X_i). \tag{12}$$

Given a scenario with multiple model outputs and sufficient data, the likelihood of factorizing over the outputs is defined. $H_\theta(X)$ is referred to as our sufficient statistics, and the multi-task likelihood is calculated as follows:

$$\begin{aligned} p\left(\hat{y}_1, \hat{y}_2 \mid H_\theta(X_i)\right) &= p\left(\hat{y}_1 \mid H_\theta^1(X_i)\right) \cdot p\left(\hat{y}_2 \mid H_\theta^2(X_i)\right) \\ &= \mathcal{N}\left(\hat{y}_1; H_\theta^1(X_i), \sigma_1^2\right) \cdot \mathcal{N}\left(\hat{y}_2; H_\theta^2(X_i), \sigma_2^2\right), \end{aligned} \tag{13}$$

where the parameter σ_1^2 and σ_2^2 are the observed noise, and $\mathcal{N}$ is the log-likelihood, written as

$$\log p\left(\hat{y}_1 \mid H_1^\theta(X_i)\right) \propto -\frac{1}{2\sigma_1^2} \parallel \hat{y}_1 - H_1^\theta(X_i) \parallel^2 - \log\sigma_1, \tag{14}$$

$$\log p\left(\hat{y}_2 \mid H_2^\theta(X_i)\right) \propto -\frac{1}{2\sigma_2^2} \parallel \hat{y}_2 - H_2^\theta(X_i) \parallel^2 - \log\sigma_2. \tag{15}$$

This brings up the minimization objective, $\mathcal{L}(\theta, \sigma_1, \sigma_2)$, (sharing-loss), which can be defined as follows:

$$\begin{aligned} &= -\log p\left(\hat{y}_1, \hat{y}_2 \mid H_\theta(X_i)\right) \propto \frac{1}{2\sigma_1^2} \parallel \hat{y}_1 - H_1^\theta(X_i) \parallel^2 \\ &\quad + \frac{1}{2\sigma_2^2} \parallel \hat{y}_2 - H_2^\theta(X_i) \parallel^2 + \log\sigma_1\sigma_2. \end{aligned} \tag{16}$$

Minimizing the last objective with respect to σ_1 and σ_2 requires learning the relative weight of the losses $\mathcal{L}_{\sigma_1}(\theta)$ and $\mathcal{L}_{\sigma_2}(\theta)$ adaptively, and should be adaptive to the data. As σ_1, which is the noise for $\hat{y}_1$, increases, the weight of $\mathcal{L}_{\sigma_1}(\theta)$ decreases. Meanwhile, the weight of the relevant objective rises as the noise level drops. The last term in the objective, which serves as a regularizer for the noise terms, should not, however, cause the noise to increase excessively because this could cause the objective to ignore the data.

3.4 Stream-Based Active Learning

A drift detection model can be obtained in our proposed offline training process. However, when using the model to detect real-world data streams, the model can still be fine-tuned to improve

the performance because it is trained based on a small dataset. Unfortunately, it is difficult to fine-tune the model because we do not know the true type and location of concept drift present in the real-world data streams. Although the true type and location can be labeled manually, the cost of labeling will be too expensive if this information is inputted manually and sample s_i is collected simultaneously. Hence, to fine-tune the drift detection model, only the best samples for labeling should be maintained.

We suggest an entropy-based active learning approach [17, 34] that includes two selection strategies for drift type identifier and drift point locator, respectively. The drift type identifier represents the probability $P(k)$ that sample s_i belongs to class k, where $\sum_{k \in K} P(k) = 1$. Thus, the entropy of the classification is

$$H(K) = -\sum_{k \in K} P(k) \log p(k). \tag{17}$$

When the entropy $H(k) > 0.5$, the selection strategy is triggered. However, the entropy strategy cannot be used directly by the drift point locator because it is designed as a regression task. To deal with the problem, a passive sampling approach is used as a selection strategy [40]. First, we assume the number of unlabeled samples is $\hat{N}$ and the number of labeled samples is m in the pool, then we use A_0 to indicate features count in the input space and label the first A_0 samples.

Suppose we have already used outputs $\{z_m\}_{m=1}^{a}$ to label the first $a(a \geq A_0)$ samples, and we have constructed a regression model $f_{locator}(X)$. For each of the remaining $\hat{N} - a$ unlabeled samples $\{X_n\}_{n=a+1}^{\hat{N}}$, the regression strategy computes the distance between the first y_m and each of the a outputs:

$$d_{nm}^{z} = \|f(X_n) - z_m\|, m = 1, \ldots, a; n = a+1, \ldots, \hat{N}. \tag{18}$$

Then we select to label the sample with the maximum d_n^z. The d_n^y is the shortest distance from $f(X_n)$ to $\{z_m\}_{m=1}^{a}$:

$$d_n^z = \min_m d_{nm}^z, n = a+1, \ldots, N. \tag{19}$$

The selection strategy of the two tasks is an "or" relationship in that, as long as either strategy is triggered, no label data will be returned. The pseudo-code of our proposed SAL is provided in Algorithm 3.

4 EXPERIMENTS

Due to the aim of this research work being to discuss the impact of the drift type on drift adaptation, we conducted three experiments to evaluate Type-LDA. The purpose of the first studies, which are covered in Section 4.1, was to evaluate the effectiveness of Type-LDA in detecting the type and timing of concept drift. Section 4.2, using toy datasets with known drifts, compares the performance of Type-LDA and other drift detection or adaptation techniques. Section 4.3 assesses Type-LDA's advantages and disadvantages on real-world datasets without known drifts. Additionally, we will not go into detail about how various learning models differ from one another, such as **Gaussian Naive Bayes** (**GaussianNB**) or Decision Tree. Given its quick processing time and partial fit option, GaussianNB from sklearn [29] was chosen as the default learning model for all experiments. We realize the implementation of Type-LDA using PyTorch and assess its performance on a 10-core CPU (i7-12700F) and a GPU with 8 GB of memory (RTX-3060Ti). Code has been released on the public GitHub repository at https://github.com/liaub/Type-LDA.

ALGORITHM 3: Stream-Based Active Learning.

Input : sample set $S_{\hat{N}}$, which contains a labeled sample set $S_{\hat{N}}^{labeled} = \{(X_i, Y_i)\} = \{(X_i, (y_i, z_i))\}, i \in [1, m]$, and a unlabeled sample set $S_{\hat{N}}^{unlabeled} = \{X_i\}, n \in [m+1, \hat{N}]$.

Parameter: X_i is the obtained error-features at time- stamp i, y_i is the drift type label and z_i is the drift point label, $\hat{N}$ is the total number of sample set, m = RANDOM($\hat{N} * 0.2$) is the number of test datasets labeled in advance by the experts. Get the test samples have already been labeled with drift point outputs $\{z_n\}_{n=1}^{m}$. $p_i(y_i = k \mid X_i)$ is probability for each drift type. DUPLICATE(Q) is the function of removing duplicate samples from the set Q.

Output : The Type-LDA model $f(X)$.

for *i in* $\{m, \ldots, \hat{N}\}$ **do**
  Using s_i to compute $p_i(y_i = k \mid X_i)$ in (4)
  $H_i^{\hat{N}-m}(K) = -\sum_{k \in K} p_i(y_i = k \mid X_i) \log p_i(y_i = k \mid X_i)$
  Using s_i to compute $\hat{z}$ in (9)
  for *j in* $\{1, \ldots, m\}$ **do**
    $d_{ij}^z = \|\hat{z} - z_j\|$
    $d_i^z = \min_j d_{ij}^z$
  end
end
Reverse order $d_{\hat{N}-m}^z$, select the first M samples, add to set Q
Sort $H_{\hat{N}-m}(K)$, select the first M samples, add to set Q
$\tilde{Q}$ = DUPLICATE(Q)
return set $\tilde{Q}$ and experts labeling
Incremental training the Type-LDA model $f(X)$ from set $\tilde{Q}$.

4.1 Concept Drift Detection

Experiment 1. Drift type identifier on data streams with different deep structures of prototypical-networks

The quality of the detector in Type-LDA is dependent on the various deep structures of the prototypical networks and error features in the data stream. Hence, in this experiment, we looked specifically at how alternative deep prototypical-network architectures affected learning performance. The idea is to create a number of data streams with predetermined forms of concept drift and then apply Type-LDA to those streams with various types of prototypical networks while using the same error features to observe the outcomes. We expect to see that the detector will be able to classify the concept drift by the error features in the data stream.

Experiment Settings In order to simulate a learning model's prequential prediction errors, we used the MOA tool to create a total of 20,000 data streams. Each data stream had 5,000 instances and contained only one type of drift. In this article, we can detect four types of drift: sudden, gradual, incremental, and normal (no drift). Then, on each type of concept drift, we chose 4,000 data streams—or a total of 16,000 data streams—as a training set for extracting drift features. The testing sets were obtained by mixing the remaining data streams. This implies a training-to-testing dataset ratio of 8:2. Four embedding functions were chosen as the non-linear mapping for the architecture of the prototypical neural network: a full attention mechanism neural network (FAN), a **fully convolutional network** model (**FCN**), a **recurrent neural network** model (**RNN**), and a 4-layered **fully connected neural network** (**FNN**). For the parameters in error-features X_i, error-features count l and sampling frequency n pair was set to $\langle 50, 25 \rangle$. For each concept drift

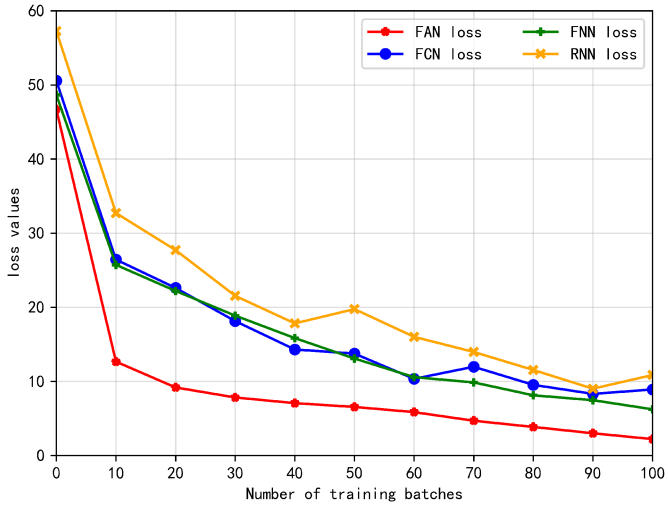


Fig. 5. Training loss of different deep structures of prototypical networks.

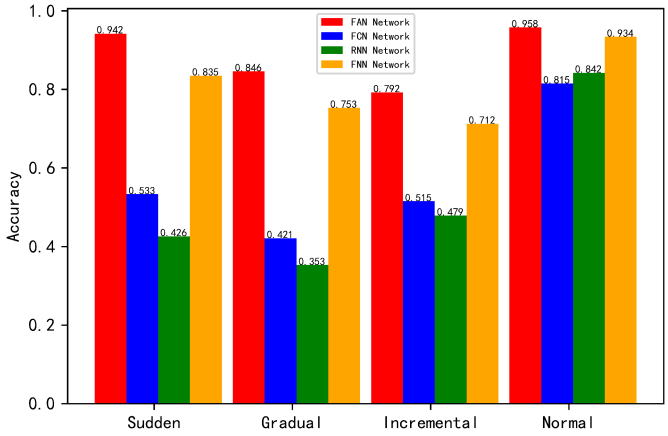


Fig. 6. Accuracy of different deep structures of prototypical networks.

category in the training phase, five support data streams were randomly selected to serve as the prototype. To assess the model's efficacy, 15 query data streams were chosen, and 20 support data streams from the training dataset were chosen at random to function as the final prototype during the testing phase. A ReLU activation was applied in all experiments, and the distance function was Euclidean. All the models were trained via SGD [33] with Adam [19] optimizer at a learning rate of 0.01.

Findings and Discussion A series of loss values can reflect the "health" of the network during training. In Figure 5, with each batch of training, all networks begin to learn concept drift characteristics to varying degrees; we found that the loss of FAN network structure tends to decline steadily, and the final loss is minimal, showing good robustness. Figure 6 shows the results of Experiment 1. The figure shows that Type-LDA was successful in detecting the simulated idea drifts. In summary, the results were as anticipated, with the FAN variation performing the best since the attention mechanism was better able to identify error-features in the data streams. In addition, the

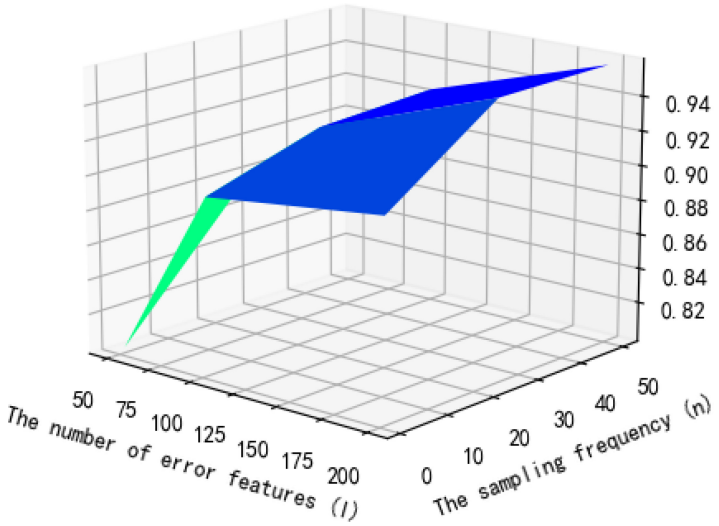


Fig. 7. Accuracy of different error features.

results show that even if the settings were the same, the performance of each type of concept drift was greatly different. Sudden drifts were easier to identify.

Experiment 2. Drift type identifier on data streams with different error-features

The impact of error-features on learning performance is the subject of our next investigation, which will further highlight the detector's quality. With Type-LDA, extracting error-features depends on l and n, where l indicates error-features count, and n indicates the sampling frequency. Hence, intuitively, we are about to create a collection of data streams with a pre-defined concept drift type, and analyze the results when Type-LDA is applied to these streams with a different error-features count l and different sampling frequencies n.

Experiment Settings. We chose the same parameter settings and learning model for Experiment 2 as for Experiment 1. However, there were 10,000 instances and four different forms of drift in the testing data stream. Considering the outcomes of Experiment 1, we implemented a deep FAN structure as the prototypical network. However, to discover the influence of l and n when extracting the error features, we kept the error-features count l changing within $l_{vaild} \in \{50, 100, 200\}$. We also kept the sampling frequency n changing within $n_{vaild} \in \{1, 25, 50\}$.

Findings and Discussion. The results of Experiment 2 are shown in Figure 7. There are two noteworthy points. The first is that the prediction accuracy is influenced by the error-features count l and the sampling frequency n. However, these relationships are not identical. When n and l increases, the prediction accuracy increases, but the increase is not infinite. When n and l reach a certain level, the accuracy stabilizes. However, every setting achieves an accuracy of more than 87% except the set of $< n = 1, l = 50 >$. Even with the set of $< n = 1, l = 50 >$, it obtained an accuracy of more than 80%. Therefore, our method is robust to the sizes of the window.

Experiment 3. Drift point locator on data streams with different error-features

We focused on investigating the drift point locator performance in this experiment. To do this, we ran two tests: one with the prototype vector and one without. We postulate that the prototype vector contains information that is very helpful for predicting the drift point; thus, we expect to see improved drift point prediction when using the prototype vector.

Experiment Settings. Based once more on the results of Experiment 1, we implemented the prototype vector in the FAN variant. However, to investigate the influence of l and n when

Table 1. The Results Details of Drift Point Locator

$l-n$	prototype-regression		normal-regression	
	Training	Testing	Training	Testing
50 - 1	82.05%	35.32%	81.92%	31.20%
50 - 25	98.97%	90.42%	94.55%	88.46%
50 - 50	98.18%	95.68%	99.34%	95.21%
100 - 1	83.72%	44.46%	81.60%	35.06%
100 - 25	98.75%	93.61%	99.82%	92.00%
100 - 50	99.92%	97.84%	99.96%	96.23%
200 - 1	90.20%	62.10%	95.64%	58.46%
200 - 25	**99.97**%	98.14%	**99.98**%	97.48%
200 - 50	**99.97**%	**99.03**%	**99.98**%	**98.15**%

[1]prototype-regression is a regression model with error-features.
[2]normal-regression is a regression model without error-features.

Table 2. The Results Details of SAL

Drift Type	F1-Score		Accuracy	
	Type-LDA	Type-LDA$^+$	Type-LDA	Type-LDA$^+$
Sudden drift	0.72	0.86	82.97%	89.44%
Gradual drift	0.69	0.82	81.41%	87.16%
Incremental drift	0.63	0.76	71.03%	77.93%
Normal	0.78	0.89	88.69%	91.25%

[1]Type-LDA: Drift type identifier + Drift point locator.
[2]Type-LDA$^+$: Type-LDA + our proposed SAL method.

predicting the drift point, the error-features count l and the sampling frequency n was changed within the set $l_{vaild} \in \{50, 100, 200\}$ and $n_{vaild} \in \{1, 25, 50\}$, respectively. We then constructed two regression prediction models—one that considered the error-features (prototype-regression) and one that did not (normal-regression). These were used to explore the influence of error-features on the drift point locator.

Findings and Discussion. Table 1 lists the results of Experiment 3. Table 1 shows that error-features count l and the sampling frequency n both affect prediction accuracy, but our method is robust to these two parameters. In addition, under the same conditions for n and l, the drift point prediction accuracy of the normal-regression is lower than the prototype-regression. With a small n and l, prototype-regression gives full play to its advantages, and the drift point prediction accuracy is significantly better than with normal-regression. However, as n and l increase, both regression models learn the characteristics of the drift points from the samples, so both models work well. Based on the result, we can conclude the drift point information contained in the error-features can help the regression model achieve better results.

Experiment 4. Active learning module

We focused on investigating the performance of SAL method in this experiment. To do this, we ran two tests—one with the SAL method (Type-LDA$^+$) and one wihtout (Type-LDA). We postulate the SAL method will help improve the prediction performance. We expect to see improved drift point prediction when using the SAL method.

Experiment Settings. To pre-train the Type-LDA, we chose the same parameter settings and learning model as for Experiment 1, but Type-LDA$^+$ will not be refined by our proposed

stream-based active method when new data streams are detected. For generating test data streams, we utilized the MOA tool once again to generate four types of test data streams, including sudden drift, gradual drift, incremental drift, and non-drift. In contrast to Experiment 1, modifications were made to the random strategy during the generation process, resulting in new drift data. We chose 100 data streams, which amounts to a total of 400 data streams, for fine-tuning the active learning. The testing dataset was derived by mixing the remaining data streams. This implies an 8:2 ratio between the fine-tuning and testing datasets. F1-score was used as our evaluation indicator, and was defined as the equilibrium between accuracy and recall. Accuracy was defined as the proportion of positive results that were actually values. Recall was defined as a measure of the model's ability to recognize positive examples; that is, the proportion of predicted positive samples to actual positive samples. There were four drift types, and the F1-score was calculated as the average of the four types:

$$\mathrm{F}_1 = \frac{\sum_{n=4}^{k} \left[2 * \left(P_k * R_k\right) / \left(P_k + R_k\right)\right]}{n}, \tag{20}$$

where P_k is the accuracy for each class k and R_k is the recall for each class k.

Findings and Discussion. To ensure that our experimental results were robust, we repeated the experiment 10 times. Table 2 lists the average F1-score and prediction accuracy. We can see from the results of Experiment 4 that the F1-score and prediction accuracy of Type-LDA$^+$ is greater than that of Type-LDA when data streams have concept drift. Based on these results, we can conclude the SAL method can help our proposed drift detector achieve better results.

4.2 Stream Learning on Toy Datasets with Simulated Concept Drift

Experiment 5. Drift detection on toy datasets.

We focused on comparing the performance of Type-LDA to several recent DDMs in this experiment. However, because these comparators cannot classify the type of drift, we only compared the prediction errors in extreme cases. To do this, we generated a number of synthetic data streams with pre-specified drift types, frequencies and intensities in order to replicate the consequential prediction errors of a learning model using various drift detection techniques. Fewer prediction errors lead to better performance. In addition, our experiment has two baselines. One baseline does not consider the concept drift, and we only use the GuassianNB algorithm (* represents the GuassianNB algorithm); the second baseline is the DDM algorithm because it is the most popular drift detection algorithm.

Experiment Settings. We chose the same parameter settings and learning model as for Experiment 1, and we chose the MOA tool to generate the toy datasets, which are the SEA, HYP, AGR, RBF, and RTG generators.

Although we also utilize the SEA and HYP generator to generate the testing datasets in this experiment, the training set's data streams are totally different from those in the testing set. All generators produced 50,000 samples. In order to analyze the effectiveness of our methods, we compared two mainstream strategy, lazy and ensemble. The lazy strategy contains ADWIN [6], DDM [12], EDDM [4], HDDM-A [9], HDDM-W [9], Page-Hinkley [21], and KSWIN [31]. The ensemble strategy contains AAE [20], BLAST [37], and KUE [7]. We trained Type-LDA with the parameters that had the best performance in Experiment 1, setting l to 50 and n to 50 to extract the error-feature types. We conducted five separate executions for each model and then calculated the average of the maximum and minimum accuracy values as the test accuracy. We use the symbol "±" to denote the difference between the test accuracy and the range of maximum and minimum values, which represents the floating value.

Table 3. Experiments on Toy Datasets

Models	SEA	HYP	AGR	RBF	RTG
*	80.1 ± 1.3	73.0 ± 1.5	73.3 ± 1.2	65.4 ± 1.2	60.2 ± 1.4
*+ADWIN	81.3 ± 1.7	73.0 ± 1.5	76.3 ± 1.9	65.3 ± 1.2	60.5 ± 1.4
*+DDM	80.7 ± 1.7	78.1 ± 2.1	74.2 ± 1.9	67.5 ± 1.4	60.5 ± 1.9
*+EDDM	81.0 ± 1.5	75.1 ± 1.8	68.1 ± 1.3	63.8 ± 1.5	61.2 ± 1.6
*+HDDM-A	81.7 ± 1.2	73.6 ± 1.4	75.3 ± 1.1	65.3 ± 1.6	60.5 ± 1.5
*+HDDM-W	81.5 ± 1.8	74.4 ± 2.2	75.4 ± 1.7	65.8 ± 1.6	62.3 ± 1.8
*+Page-Hinkley	82.0 ± 1.2	73.6 ± 1.4	74.4 ± 1.5	65.3 ± 1.4	60.5 ± 1.1
*+KSWIN	81.1 ± 1.5	76.2 ± 1.8	74.5 ± 2.1	65.9 ± 1.4	61.2 ± 1.5
*+AAE	80.5 ± 1.0	76.2 ± 1.8	74.8 ± 2.1	64.9 ± 1.5	60.8 ± 1.6
*+BLAST	81.3 ± 1.3	74.6 ± 1.6	75.5 ± 1.2	65.5 ± 1.5	61.5 ± 1.5
*+KUE	81.7 ± 1.2	78.5 ± 1.6	77.5 ± 1.8	66.7 ± 1.2	61.5 ± 1.2
*+Type-LDA	82.1 ± 0.8	82.7 ± 1.1	73.2 ± 1.3	69.4 ± 1.1	63.4 ± 0.9
*+Type-LDA$^+$	**83.2** ± 0.6	**86.6** ± 1.2	**88.8** ± 1.1	**70.9** ± 0.8	**66.8** ± 1.0

[1]The experiments were evaluated using accuracy in percentage (%).
[2] * represents the GaussianNB algorithm. ± means the floating value of accuracy.
[3]Type-LDA$^+$ refers to the model obtained by fine-tuning Type-LDA in active learning way.

Table 4. F1-Score on Toy Datasets

Models	SEA	HYP	AGR	RBF	RTG
ADWIN	0.62	Null	0.50	Null	Null
DDM	Null	0.63	0.50	0.66	Null
EDDM	0.38	0.63	0.25	0.38	0.38
HDDM-A	0.69	Null	0.63	Null	Null
HDDM-W	0.63	0.70	0.63	0.50	0.55
Page-Hinkley	0.71	Null	0.50	Null	Null
KSWIN	0.38	0.66	0.50	0.63	0.38
AAE	0.34	0.62	0.48	0.58	0.36
BLAST	0.62	0.68	0.65	0.54	0.53
KUE	0.66	0.70	0.56	0.61	0.58
Type-LDA	0.72	0.75	0.63	0.66	0.67
Type-LDA$^+$	**0.86**	**0.87**	**0.89**	**0.68**	**0.75**

[1]Null indicates that no drifts have been detected.
[2]Type-LDA$^+$ refers to the model obtained by fine-tuning Type-LDA in active learning way.

Findings and Discussion. Tables 3 and 4 show the results of Experiment 5. In general, we can observe that integrating any of the comparative drift detection approaches improves the performance of any classifier in almost all toy data streams. This is true despite the fact that we just utilized a simple drift adaption method, in which a new classifier is learned once a drift point is predicted. Overall, comparing all toy data streams, the prediction accuracy of Type-LDA was superior to that of the other DDMs. Especially on the AGR dataset, the performance of our algorithm is at least 10% higher than others' algorithms. From Table 4, we can see the F1-score also improved significantly using our proposed method, and concept drift on each of the data streams can be detected by our method. Hence, the results further prove a drift detector with good performance

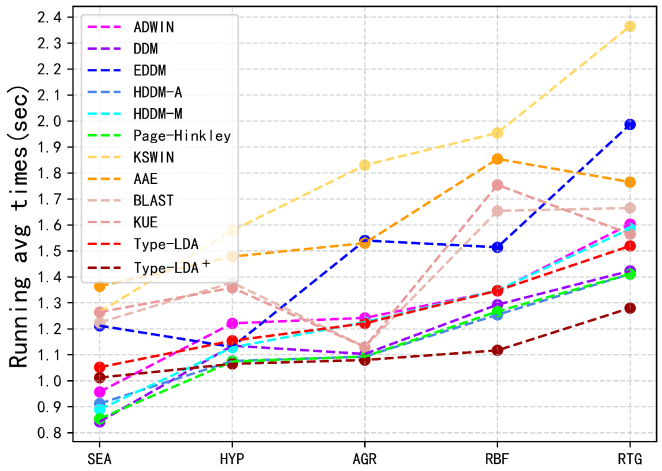


Fig. 8. Running avg times (sec) based on toy datasets.

can be learnt by an offline trained process. Moreover, comparison of the last two rows in Table 3 and Table 4 demonstrates that the SAL method can help our proposed drift detector achieve better results.

In addition, we analyzed the detection times of each DDM to further highlight the advantages of Type-LDA. The results are listed in Figure 8, and we can see that Type-LDA has a substantial performance in terms of detection time. This outcome validates our analysis of the LDA's time complexity. Its order of time complexity is comparable to that of the current drift detection techniques.

4.3 Stream Learning on Real-World Datasets

Experiment 6. Drift detection on real-world datasets

In this experiment, we investigated the performance of Type-LDA on real-world datasets. However, as real-world data streams have unknown concept drifts, we cannot calculate the F1 score. Like most previous DDMs [38, 43, 49], we only use the prediction accuracy to illustrate the advantages of each algorithms. We hold that such a comparison is reasonable. After all, the ultimate goal of the concept DDMs is to ensure that the learner's accuracy is not affected by the concept drift.

Experiment Settings. The basic settings mirrored those of Experiment 5. The primary distinction is that we employed a collection of real-world datasets with unknown drifts, which are frequently used to evaluate the effectiveness of drift detection techniques:

— **Elec:** the Electricity Price Dataset contains 45,325 samples, each with 9 features.
— **Weather:** the NAOO Weather Dataset contains 18,712 samples, each with 9 features.
— **Spam:** the Spam Filtering Dataset comprises 9829 samples, each with 501 features.
— **Air:** the Airline Delay Dataset has 539,395 samples with 8 features.
— **Poke:** This dataset consists of 1,000,000 instances and 11 attributes.

We set parameters l and n to 50 and 10, respectively, in the Elec, Weather, and Spam datasets. In the other datasets, we set parameters l and n to 100 and 25, respectively.

Findings and Discussion. Table 5 provides a comparison of the learning accuracy of the various drift detection techniques. At first sight, we get the same results as with the toy datasets, namely, that, generally speaking, relearning a new classifier after a drift is detected improves classification accuracy. In addition, Type-LDA continues to achieve the best performance across all

Table 5. Experiment Results on Real-World Datasets

Models	Elec	Weather	Spam	Air	Poke
*	73.9 ± 1.1	68.7 ± 1.8	89.3 ± 1.3	58.1 ± 1.9	67.0 ± 1.5
*+ADWIN	79.1 ± 1.6	69.5 ± 1.5	92.4 ± 1.7	62.2 ± 2.0	64.2 ± 2.2
*+DDM	82.1 ± 2.1	70.3 ± 2.2	91.9 ± 1.8	61.9 ± 1.7	64.2 ± 2.6
*+EDDM	82.6 ± 1.7	70.6 ± 1.9	92.5 ± 1.6	61.5 ± 1.9	64.2 ± 1.5
*+HDDM-A	83.1 ± 1.4	71.3 ± 1.8	92.1 ± 1.5	62.6 ± 1.8	64.5 ± 2.4
*+HDDM-W	82.7 ± 2.5	72.6 ± 2.1	92.2 ± 2.4	61.5 ± 2.1	62.0 ± 1.7
*+Page-Hinkley	78.2 ± 1.2	69.6 ± 1.6	91.8 ± 1.4	62.8 ± 1.8	64.1 ± 1.8
*+KSWIN	81.4 ± 2.3	67.8 ± 1.1	92.4 ± 2.5	61.3 ± 1.4	63.6 ± 1.7
*+AAE	80.5 ± 1.8	68.8 ± 1.6	92.6 ± 1.8	61.4 ± 1.5	63.4 ± 1.8
*+BLAST	80.8 ± 1.6	70.5 ± 1.8	92.5 ± 1.6	61.6 ± 1.8	64.5 ± 1.6
*+KUE	81.5 ± 2.0	69.5 ± 1.8	92.5 ± 1.5	62.5 ± 1.6	64.8 ± 1.8
*+Type-LDA	82.3 ± 1.0	70.5 ± 0.7	92.8 ± 0.6	60.9 ± 0.9	65.7 ± 1.6
*+Type-LDA$^+$	**86.2** ± 0.8	**74.7** ± 0.5	**94.1** ± 0.6	**65.0** ± 1.1	**72.4** ± 1.2

[1]The experiments were evaluated using accuracy in percentage (%).
[2] * represents the GaussianNB algorithm. ± means the floating value of accuracy.
[3]Type-LDA$^+$ refers to the model obtained by fine-tuning Type-LDA using actvie learning.

datasets, despite the fact that it does not detect the majority of drifts. This outcome demonstrates once more that LDA is the most accurate at detecting drift. There are, in our opinion, two reasons for this. Firstly, some methods have a limited capacity for drift detection, hence we cannot detect some drifts present in some real-world datasets by these DDMs. For instance, ADWIN, DDM, and Page-Hinkley do not detect as many drifts in the Weather dataset as the other methods. Therefore, we cannot detect many drifts using these methods; i.e., they are not very accurate. By contrast, Type-LDA and EDDM detected more drifts, which means their accuracy was better. Secondly, some DDMs have limitations that cause the incorrect drifts to be detected. For instance, in the Poker dataset, HDDM-W detected the most drifts but did not have the highest accuracy. This indicates some distributions were falsely detected as drifts.

Additionally, we contrasted the number of drifts discovered using various parameters l and n. The experimental results are listed in Table 6. As l and n increase, the drifts detected count decreases. For instance, in the Elec dataset, with l fixed at 50, the detected drifts count decreases from 482 to 130 as n increases from l to 25.

At the same time, we compared each algorithm in terms of its running time. The results, shown in Table 7, demonstrate that Type-LDA or Type-LDA$^+$ still has a better performance than the other drifts detection methods. Unlike the results of Figure 8, the inference speed of Type-LDA or Type-LDA+ seems to be superior on real-world data streams.

Finally, we investigated the performance of active learning module we proposed. Although a comparison between the two final lines of Table 3 and Table 5 demonstrates that active learning improved the performance of Type-LDA. We also show the accuracy over time in Figure 9. From Figure 9, we can see that the performance of Type-LDA+ is lower than Type-LDA at the beginning of the training time because the model has not adapted to the interactive information. But over time, we can see the effectiveness of our proposed SAL method.

5 CONCLUSIONS AND FUTURE WORK

To discuss the impact of the drift type on drift adaptation, we proposed a new drift adaptation framework called Type-LDA to extract the "when" and "how" details related to concept drift. To

Table 6. The Number of Detected Drifts Using Different Settings of Parameters l and n

	Datasets	n = 1	n = 25	n = 50
l = 50	Elec	482	260	130
	Weather	167	95	42
	Spam	58	50	16
	Air	254	233	128
	Poke	6002	5064	2645
l = 100	Elec	436	122	81
	Weather	146	98	36
	Spam	53	28	14
	Air	26	41	3
	Poke	3591	1679	1426
l = 200	Elec	427	98	52
	Weather	142	94	27
	Spam	47	29	21
	Air	127	43	38
	Poke	2615	1003	876

Table 7. Running Avg Times (Sec) Based on Real-World Datasets

Models	Elec	Weather	Spam	Air	Poke
ADWIN	0.648	1.281	2.021	4.294	497.110
DDM	0.337	2.987	0.809	93.088	413.490
EDDM	0.345	0.326	0.840	2.001	395.920
HDDM-A	0.329	0.447	1.393	2.128	102.198
HDDM-W	0.370	0.378	2.710	0.575	0.836
Page-Hinkley	1.331	1.796	4.890	3.942	26.893
KSWIN	0.406	0.340	1.556	0.623	1.717
AAE	0.502	1.245	2.352	2.325	3.518
BLAST	0.458	1.122	1.956	2.626	3.765
KUE	0.402	0.545	1.354	1.825	1.613
Type-LDA	**0.204**	**0.191**	**0.506**	1.878	1.592
Type-LDA$^+$	0.328	0.488	0.8583	**0.438**	**1.462**

detect concept drift, error features were extracted based on the error rates of various concept drifts. Then, a drift type identifier and a drift point locator are built to extract the "when" and "how" details, respectively. During the adaptation phase, these details related to concept drift can be used to guide the re-learning of the new model, so that it can adjust to the relevant data stream. Therefore, unlike the present drift adaptation methods, although Type-LDA needs to detect concept drift, it does not rely on statistic tests set by humans to judge whether concept drift has occurred. Rather, it proves that an end-to-end model can be learned to detect concept drift automatically. Numerous

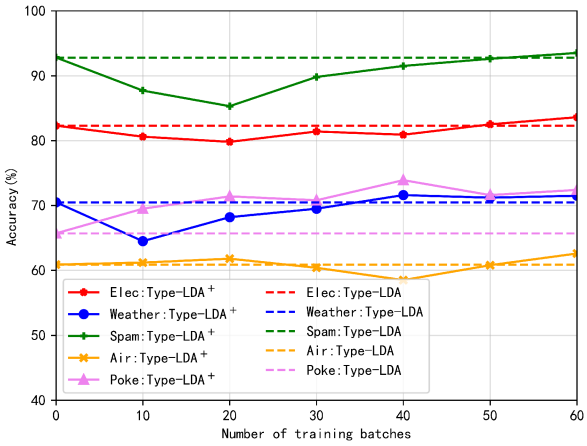


Fig. 9. The accuracy over time.

tests using simulated and real-world datasets demonstrate that Type-LDA performs better than a number of the most popular drift adaptation techniques.

In our proposed Type-LDA framework, the SAL method can further improve the performance of Type-LDA, but it needs to manually label the type of concept drift. Hence, in the future work, we aim to propose an automatic active learning method to minimize interaction.

REFERENCES

[1] Ahmad Abbasi, Abdul Rehman Javed, Chinmay Chakraborty, Jamel Nebhen, Wisha Zehra, and Zunera Jalil. 2021. ElStream: An ensemble learning approach for concept drift detection in dynamic social big data stream learning. *IEEE Access* 9 (2021), 66408–66419.

[2] Supriya Agrahari and Anil Kumar Singh. 2021. Concept drift detection in data stream mining: A literature review. *Journal of King Saud University-Computer and Information Sciences* 34, 10 (2022), 9523–9540.

[3] Rakesh Agrawal, Tomasz Imielinski, and Arun Swami. 1993. Database mining: A performance perspective. *IEEE Transactions on Knowledge and Data Engineering* 5, 6 (1993), 914–925.

[4] M. Baena-Garc, J. D. Campo-Ávila, R. Fidalgo, A. Bifet, and R. Morales-Bueno. 2006. Early drift detection method. In *Proceedings of the International Workshop on Knowledge Discovery from Data Streams*.

[5] Manuel Baena-Garcıa, José del Campo-Ávila, Raúl Fidalgo, Albert Bifet, R. Gavalda, and Rafael Morales-Bueno. 2006. Early drift detection method. In *Proceedings of the 4th International Workshop on Knowledge Discovery from Data Streams*. Vol. 6, 77–86.

[6] Albert Bifet and Ricard Gavalda. 2007. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*. SIAM, 443–448.

[7] Alberto Cano and Bartosz Krawczyk. 2020. Kappa updated ensemble for drifting data stream mining. *Machine Learning* 109, 1 (2020), 175–218.

[8] Ryan Elwell and Robi Polikar. 2011. Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks* 22, 10 (2011), 1517–1531.

[9] Isvani Frias-Blanco, Jose Del Campo-Avila, Gonzalo Ramos-Jimenez, Rafael Morales-Bueno, Agustin Ortiz-Diaz, and Yaile Caballero-Mota. 2015. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering* 27, 3 (2015), 810–823.

[10] Isvani Frias-Blanco, José del Campo-Ávila, Gonzalo Ramos-Jimenez, Rafael Morales-Bueno, Agustin Ortiz-Diaz, and Yaile Caballero-Mota. 2014. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering* 27, 3 (2014), 810–823.

[11] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. 2004. Learning with drift detection. In *Proceedings of the Brazilian Symposium on Artificial Intelligence*. Springer, 286–295.

[12] J. Gama, P. Medas, G. Castillo, and P. P. Rodrigues. 2004. Learning with drift detection. In *Brazilian Symposium on Artificial Intelligence*. Springer, 286–295.

[13] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM Computing Surveys* 46, 4 (2014), 1–37.
[14] Husheng Guo, Hai Li, Qiaoyan Ren, and Wenjian Wang. 2022. Concept drift type identification based on multi-sliding windows. *Information Sciences* 585 (2022), 1–23.
[15] Ben Halstead, Yun Sing Koh, Patricia Riddle, Russel Pears, Mykola Pechenizkiy, Albert Bifet, Gustavo Olivares, and Guy Coulson. 2022. Analyzing and repairing concept drift adaptation in data stream classification. *Machine Learning* 111, 10 (2022), 3489–3523.
[16] Geoff Hulten, Laurie Spencer, and Pedro Domingos. 2001. Mining time-changing data streams. In *Proceedings of the 7th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 97–106.
[17] Dino Ienco, Albert Bifet, Indrė Žliobaitė, and Bernhard Pfahringer. 2013. Clustering based active learning for evolving data streams. In *Proceedings of the International Conference on Discovery Science*. Springer, 79–93.
[18] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 7482–7491.
[19] D. Kingma and J. Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
[20] Bartosz Krawczyk. 2017. Active and adaptive ensemble learning for online activity recognition from data streams. *Knowledge-Based Systems* 138 (2017), 69–78.
[21] Anjin Liu, Jie Lu, Yiliao Song, Junyu Xuan, and Guangquan Zhang. 2022. Concept drift detection delay index. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4585–4597.
[22] Anjin Liu, Guangquan Zhang, and Jie Lu. 2017. Fuzzy time windowing for gradual concept drift adaptation. In *Proceedings of the 2017 IEEE International Conference on Fuzzy Systems*. IEEE, 1–6.
[23] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang. 2020. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering* 31, 12 (2018), 2346–2363, 2018, 1–1.
[24] Yang Lu, Yiu-Ming Cheung, and Yuan Yan Tang. 2019. Adaptive chunk-based dynamic weighted majority for imbalanced data streams with concept drift. *IEEE Transactions on Neural Networks and Learning Systems* 31, 8 (2019), 2764–2778.
[25] Kyosuke Nishida and Koichiro Yamauchi. 2007. Detecting concept drift using statistical testing. In *Proceedings of the 10th International Conference on Discovery Science*.
[26] Gustavo Oliveira, Leandro L. Minku, and Adriano L. I. Oliveira. 2021. Tackling virtual and real concept drifts: An adaptive Gaussian mixture model approach. *IEEE Transactions on Knowledge and Data Engineering* 1 (2021), 1–1.
[27] Nikunj C. Oza and Stuart Russell. 2001. Experimental comparisons of online and batch versions of bagging and boosting. In *Proceedings of the 7th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 359–364.
[28] Sankar K. Pal and Sushmita Mitra. 1992. Multilayer perceptron, fuzzy sets, classifiaction. *IEEE Transactionson Neural Networks* 3, (1992), 683–697.
[29] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in python. *The Journal of Machine Learning Research* 12 (2011), 2825–2830.
[30] Mahardhika Pratama, Jie Lu, Edwin Lughofer, Guangquan Zhang, and Meng Joo Er. 2016. An incremental learning of concept drifts using evolving type-2 recurrent fuzzy neural networks. *IEEE Transactions on Fuzzy Systems* 25, 5 (2016), 1175–1192.
[31] C. Raab, M. Heusinger, and F. M. Schleif. 2020. Reactive soft prototype computing for concept drift streams. *Neurocomputing* 416, (2020), 340–351.
[32] Md Geaur Rahman and Md Zahidul Islam. 2022. Adaptive decision forest: An incremental machine learning framework. *Pattern Recognition* 122 (2022), 108345.
[33] David Saad. 1998. Online algorithms and stochastic approximations. *Online Learning* 5 (1998), 6–3.
[34] Jicheng Shan, Hang Zhang, Weike Liu, and Qingbao Liu. 2018. Online active learning ensemble framework for drifted data streams. *IEEE Transactions on Neural Networks and Learning Systems* 30, 2 (2018), 486–498.
[35] Jake Snell, Kevin Swersky, and Richard Zemel. 2017. Prototypical networks for few-shot learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*.
[36] Yiliao Song, Jie Lu, Haiyan Lu, and Guangquan Zhang. 2021. Learning data streams with changing distributions and temporal dependency. *IEEE Transactions on Neural Networks and Learning Systems* (2021).
[37] Jan N. van Rijn, Geoffrey Holmes, Bernhard Pfahringer, and Joaquin Vanschoren. 2018. The online performance estimation framework: Heterogeneous ensemble learning for data streams. *Machine Learning* 107, 1 (2018), 149–176.
[38] Kun Wang, Jie Lu, Anjin Liu, Yiliao Song, Li Xiong, and Guangquan Zhang. 2022. Elastic gradient boosting decision tree with adaptive iterations for concept drift adaptation. *Neurocomputing* 491 (2022), 288–304.
[39] Mingyuan Wang and Adrian Barbu. 2022. Online feature screening for data streams with concept drift. *IEEE Transactions on Knowledge and Data Engineering* 1 (2022), 1–14.

[40] Dongrui Wu, Chin-Teng Lin, and Jian Huang. 2019. Active learning for regression using greedy sampling. *Information Sciences* 474 (2019), 90–105.
[41] Shuliang Xu and Junhong Wang. 2017. Dynamic extreme learning machine for data stream classification. *Neurocomputing* 238 (2017), 433–449.
[42] Junyu Xuan, Jie Lu, and Guangquan Zhang. 2020. Bayesian nonparametric unsupervised concept drift detection for data stream mining. *ACM Transactions on Intelligent Systems and Technology* 12, 1 (2020), 1–22.
[43] En Yu, Yiliao Song, Guangquan Zhang, and Jie Lu. 2022. Learn-to-adapt: Concept drift adaptation for hybrid multiple streams. *Neurocomputing* 496 (2022), 121–130.
[44] N. Lu, G. Zhang, and J. Lu. 2014. Concept drift detection via competence models. *Artificial Intelligence*, 209 (2014), 11–18.
[45] Hang Yu, Weixu Liu, Jie Lu, Yimin Wen, Xiangfeng Luo, and Guangquan Zhang. 2023. Detecting group concept drift from multiple data streams. *Pattern Recognition* 134 (2023), 109113.
[46] Hang Yu, Jie Lu, Anjin Liu, Bin Wang, Ruimin Li, and Guangquan Zhang. 2022. Real-time prediction system of train carriage load based on multi-stream fuzzy learning. *IEEE Transactions on Intelligent Transportation Systems* 23, 9 (2022), 15155–15165.
[47] Hang Yu, Jie Lu, and Guangquan Zhang. 2020. Continuous support vector regression for nonstationary streaming data. *IEEE Transactions on Cybernetics* 52, 5 (2020), 3592–3605.
[48] H. Yu, J. Lu, and G. Zhang. 2020. An online robust support vector regression for data streams. *IEEE Transactions on Knowledge and Data Engineering* PP, 99 (2020), 1–1.
[49] Hang Yu, Jie Lu, and Guangquan Zhang. 2020. An online robust support vector regression for data streams. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2020), 150–163.
[50] Hang Yu, Jie Lu, and Guangquan Zhang. 2020. Topology learning-based fuzzy random neural networks for streaming data regression. *IEEE Transactions on Fuzzy Systems* 30, 2 (2020), 412–425.
[51] Hang Yu, Jie Lu, and Guangquan Zhang. 2021. MORStreaming: A multioutput regression system for streaming data. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 52, 8 (2021), 4862–4874.
[52] Hang Yu, Qingyong Zhang, Tianyu Liu, Jie Lu, Yimin Wen, and Guangquan Zhang. 2022. Meta-ADD: A meta-learning based pre-trained model for concept drift active detection. *Information Sciences* 608 (2022), 996–1009.
[53] Xiulin Zheng, Peipei Li, Xuegang Hu, and Kui Yu. 2021. Semi-supervised classification on data streams with recurring concept drift and concept evolution. *Knowledge-Based Systems* 215 (2021), 106749.

Received 30 November 2022; revised 28 September 2023; accepted 15 December 2023